MLA0303-Reinforcement Learning

CO-3 AT-2

Experiments

Name: S. Divya

Reg No: 192425163

1. Policy Gradient Traffic Signal Control

Experiment Objective

Design and implement a policy-based RL solution for traffic signals. The experiment uses REINFORCE vs A2C and evaluates the specified performance measures. The design is structured to satisfy the Level 4 rubric: correct concepts, systematic implementation, appropriate tools, quantitative evaluation and complete documentation.

1. RL Environment Design

- State: queue length on each approach, current signal phase, waiting vehicles, elapsed phase time and traffic density.
- Actions: select/keep the traffic-light phase. A discrete softmax policy outputs probabilities for valid phases.
- Reward: negative total waiting time with a throughput bonus. Example: r = −waiting_vehicles + 0.5×vehicles_cleared.
- Policy network: state → Dense(128, ReLU) → Dense(64, ReLU) → Softmax(action probabilities). A2C additionally uses a value network V(s).

2. Policy-Based Architecture

Environment State → Policy Network (Actor) → Action → Environment Transition → Reward → Advantage/Return Estimation → Policy Update. Where required, a critic network estimates V(s) and improves the policy-gradient update.

3. Experimental Procedure

- Create or configure the simulation environment and define observation/action spaces.
- Normalize state features and initialize actor/critic networks with reproducible random seeds.
- Train the selected algorithm for a fixed number of episodes/steps and record average vehicle waiting time, throughput, convergence rate.
- Run the same environment settings for the comparison algorithm and use identical evaluation episodes.
- Plot reward and task-specific metrics against training episodes and report mean and standard deviation over multiple seeds where possible.

4. Implementation Skeleton

```
import numpy as np, tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense

policy = Sequential([
    Dense(128, activation="relu", input_shape=(5,)),
    Dense(64, activation="relu"),
    Dense(4, activation="softmax")
])

def choose_action(state):
    probs = policy(np.asarray(state)[None,:]).numpy()[0]
    return np.random.choice(4, p=probs)

# REINFORCE update: maximize sum(log pi(a|s) * return)
# A2C update: policy advantage = return - V(s)
```

5. Algorithm Comparison

REINFORCE is simple and directly optimizes the policy, but its episode-return estimates can have high variance. A2C uses a critic to estimate V(s), producing an advantage signal that normally gives more stable learning. Evaluate both under identical traffic seeds and episode budgets.

6. Tools and Experimental Configuration

Use SUMO/TraCI or a lightweight custom traffic simulator for realistic experiments. TensorFlow or PyTorch implements the policy/value networks. Plot waiting time and throughput against training episodes.

7. Results to Record

- Primary metrics: average vehicle waiting time, throughput, convergence rate.
- Training reward curve: episode/step on the x-axis and mean reward on the y-axis.
- Convergence: identify the point at which the moving-average reward becomes relatively stable.
- Baseline comparison: compare the learned controller against the comparison algorithm and, where possible, a simple rule/random baseline.
- Validation: repeat evaluation with fixed unseen seeds/scenarios to check generalization.

8. Expected Interpretation

Expected result: A2C should generally converge more smoothly, while both methods should reduce average waiting time and increase throughput compared with an uncontrolled/fixed baseline. Exact numerical values must be reported from the actual run.

9. Conclusion

The experiment demonstrates how REINFORCE vs A2C can be applied to traffic signals. The final conclusion should be based on the actual recorded results, not assumed values. A Level 4 submission should clearly connect the observed metrics to algorithm behavior, explain convergence or instability, and justify the selected method.

10. Rubric Coverage

Understanding of Concepts (25%): RL formulation and algorithm explained. Experimental Design (30%): environment, network, training and comparison systematically defined. Use of Tools (20%): Python with TensorFlow/PyTorch and appropriate simulator/data. Analysis of Results (15%): metrics, convergence and validation. Documentation (10%): architecture, implementation, results and conclusion.

2. Actor-Critic Warehouse Robot

Experiment Objective

Design and implement a policy-based RL solution for warehouse robot. The experiment uses Vanilla Policy Gradient vs A2C and evaluates the specified performance measures. The design is structured to satisfy the Level 4 rubric: correct concepts, systematic implementation, appropriate tools, quantitative evaluation and complete documentation.

1. RL Environment Design

- State: robot position, package locations, destination, obstacle proximity, battery and current task.
- Actions: move north/south/east/west, pick package, deliver package, or wait.
- Reward: +delivery reward, −collision penalty, −step cost, and a small bonus for completing the next required task.
- Actor outputs action probabilities. Critic estimates V(s). A2C updates the actor using advantage $A(s,a)=R-V(s)$.

2. Policy-Based Architecture

Environment State → Policy Network (Actor) → Action → Environment Transition → Reward → Advantage/Return Estimation → Policy Update. Where required, a critic network estimates V(s) and improves the policy-gradient update.

3. Experimental Procedure

- Create or configure the simulation environment and define observation/action spaces.
- Normalize state features and initialize actor/critic networks with reproducible random seeds.
- Train the selected algorithm for a fixed number of episodes/steps and record cumulative reward, training stability, task completion time.
- Run the same environment settings for the comparison algorithm and use identical evaluation episodes.
- Plot reward and task-specific metrics against training episodes and report mean and standard deviation over multiple seeds where possible.

4. Implementation Skeleton

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense

actor = Sequential([
    Dense(128, activation="relu", input_shape=(8,)),
    Dense(64, activation="relu"),
    Dense(6, activation="softmax")
])
critic = Sequential([
    Dense(128, activation="relu", input_shape=(8,)),
    Dense(64, activation="relu"),
    Dense(1)
])
```

5. Algorithm Comparison

Vanilla Policy Gradient waits for complete returns and can have high variance. A2C learns a value baseline and uses the advantage to reduce variance. Compare mean reward over a fixed number of episodes, standard deviation across seeds, task completion time and success rate.

6. Tools and Experimental Configuration

Implement the warehouse environment as a grid-world first, then extend it with obstacles and multiple packages. TensorFlow/PyTorch can train actor and critic networks. Record episode reward, completion time and collision count.

7. Results to Record

- Primary metrics: cumulative reward, training stability, task completion time.
- Training reward curve: episode/step on the x-axis and mean reward on the y-axis.
- Convergence: identify the point at which the moving-average reward becomes relatively stable.
- Baseline comparison: compare the learned controller against the comparison algorithm and, where possible, a simple rule/random baseline.
- Validation: repeat evaluation with fixed unseen seeds/scenarios to check generalization.

8. Expected Interpretation

Expected result: A2C should normally provide smoother reward curves and faster task completion than vanilla policy gradient, while preserving the ability to learn a good package-collection policy.

9. Conclusion

The experiment demonstrates how Vanilla Policy Gradient vs A2C can be applied to warehouse robot. The final conclusion should be based on the actual recorded results, not assumed values. A Level 4 submission should clearly connect the observed metrics to algorithm behavior, explain convergence or instability, and justify the selected method.

10. Rubric Coverage

Understanding of Concepts (25%): RL formulation and algorithm explained. Experimental Design (30%): environment, network, training and comparison systematically defined. Use of Tools (20%): Python with TensorFlow/PyTorch and appropriate simulator/data. Analysis of Results (15%): metrics, convergence and validation. Documentation (10%): architecture, implementation, results and conclusion.

3. DDPG for Continuous Portfolio Optimization

Experiment Objective

Design and implement a policy-based RL solution for stock trading. The experiment uses Deep Deterministic Policy Gradient and evaluates the specified performance measures. The design is structured to satisfy the Level 4 rubric: correct concepts, systematic implementation, appropriate tools, quantitative evaluation and complete documentation.

1. RL Environment Design

- State: normalized asset prices/returns, volatility, current portfolio weights, cash position and recent market indicators.
- Action: continuous portfolio allocation vector. Apply softmax/normalization so weights sum to one.
- Reward: portfolio return minus transaction cost and risk penalty. Example: r = return − 0.001×turnover − λ×volatility.
- Actor maps state to continuous weights. Critic estimates Q(s,a). Target actor/critic networks and replay buffer stabilize DDPG training.

2. Policy-Based Architecture

Environment State → Policy Network (Actor) → Action → Environment Transition → Reward → Advantage/Return Estimation → Policy Update. Where required, a critic network estimates V(s) and improves the policy-gradient update.

3. Experimental Procedure

- Create or configure the simulation environment and define observation/action spaces.
- Normalize state features and initialize actor/critic networks with reproducible random seeds.
- Train the selected algorithm for a fixed number of episodes/steps and record cumulative return, risk, convergence performance.
- Run the same environment settings for the comparison algorithm and use identical evaluation episodes.

- Plot reward and task-specific metrics against training episodes and report mean and standard deviation over multiple seeds where possible.

4. Implementation Skeleton

```
import tensorflow as tf
from tensorflow.keras import layers

actor = tf.keras.Sequential([
   layers.Input((10,)), layers.Dense(128, activation="relu"),
   layers.Dense(64, activation="relu"),
   layers.Dense(3, activation="softmax")
])
critic = tf.keras.Sequential([
   layers.Input((13,)), layers.Dense(128, activation="relu"),
   layers.Dense(64, activation="relu"), layers.Dense(1)
])
```

5. Algorithm Comparison

DDPG is appropriate because portfolio allocation is continuous. The actor proposes allocations while the critic evaluates them. Exploration can be added with controlled action noise. Risk must be included in the reward rather than maximizing raw return alone.

6. Tools and Experimental Configuration

Use historical price data split chronologically into training, validation and test periods. Normalize features and include transaction costs. Evaluate cumulative return, volatility, Sharpe ratio, maximum drawdown, turnover and reward convergence.

7. Results to Record

- Primary metrics: cumulative return, risk, convergence performance.
- Training reward curve: episode/step on the x-axis and mean reward on the y-axis.
- Convergence: identify the point at which the moving-average reward becomes relatively stable.
- Baseline comparison: compare the learned controller against the comparison algorithm and, where possible, a simple rule/random baseline.
- Validation: repeat evaluation with fixed unseen seeds/scenarios to check generalization.

8. Expected Interpretation

Expected result: the learned policy should seek a favorable risk-adjusted return and avoid excessive turnover. Because financial markets are non-stationary, results must be interpreted as experimental and not as guaranteed profit.

9. Conclusion

The experiment demonstrates how Deep Deterministic Policy Gradient can be applied to stock trading. The final conclusion should be based on the actual recorded results, not assumed values. A Level 4 submission should clearly connect the observed metrics to algorithm behavior, explain convergence or instability, and justify the selected method.

10. Rubric Coverage

Understanding of Concepts (25%): RL formulation and algorithm explained. Experimental Design (30%): environment, network, training and comparison systematically defined. Use of Tools (20%): Python with TensorFlow/PyTorch and appropriate simulator/data. Analysis of Results (15%): metrics, convergence and validation. Documentation (10%): architecture, implementation, results and conclusion.

4. PPO for Smart HVAC Energy Management

Experiment Objective

Design and implement a policy-based RL solution for smart building HVAC. The experiment uses PPO vs REINFORCE and evaluates the specified performance measures. The design is structured to satisfy the Level 4 rubric: correct concepts, systematic implementation, appropriate tools, quantitative evaluation and complete documentation.

1. RL Environment Design

- State: indoor temperature, target comfort temperature, outdoor temperature, occupancy, humidity, time and recent HVAC power.
- Actions: increase/decrease HVAC control or select a bounded continuous control level.
- Reward: negative energy consumption plus a comfort penalty. Example: $r = -\text{energy\_cost} - \beta|\text{indoor\_temp} - \text{target}|$.
- PPO uses a clipped policy objective to prevent excessively large policy updates, improving training stability.

2. Policy-Based Architecture

Environment State → Policy Network (Actor) → Action → Environment Transition → Reward → Advantage/Return Estimation → Policy Update. Where required, a critic network estimates V(s) and improves the policy-gradient update.

3. Experimental Procedure

- Create or configure the simulation environment and define observation/action spaces.
- Normalize state features and initialize actor/critic networks with reproducible random seeds.
- Train the selected algorithm for a fixed number of episodes/steps and record energy consumption, occupant comfort, PPO-vs-REINFORCE performance.

- Run the same environment settings for the comparison algorithm and use identical evaluation episodes.
- Plot reward and task-specific metrics against training episodes and report mean and standard deviation over multiple seeds where possible.

4. Implementation Skeleton

```
import tensorflow as tf
from tensorflow.keras import layers

actor = tf.keras.Sequential([
   layers.Input((7,)), layers.Dense(128, activation="tanh"),
   layers.Dense(64, activation="tanh"),
   layers.Dense(3, activation="softmax")
])
critic = tf.keras.Sequential([
   layers.Input((7,)), layers.Dense(128, activation="tanh"),
   layers.Dense(1)
])

# PPO concept:
# ratio = exp(new_log_prob - old_log_prob)
# clipped_ratio = clip(ratio, 1-eps, 1+eps)
# actor_loss = -mean(min(ratio*A, clipped_ratio*A))
```

5. Algorithm Comparison

PPO constrains the policy update using a clipped objective, reducing destructive updates. REINFORCE is simpler but can have high variance. Compare both using identical initial conditions and training budgets.

6. Tools and Experimental Configuration

A simple thermal simulator can model indoor temperature using outside temperature, occupancy and HVAC control. For advanced work, use EnergyPlus/OpenStudio. Track energy use, comfort violations and convergence.

7. Results to Record

- Primary metrics: energy consumption, occupant comfort, PPO-vs-REINFORCE performance.

- Training reward curve: episode/step on the x-axis and mean reward on the y-axis.
- Convergence: identify the point at which the moving-average reward becomes relatively stable.
- Baseline comparison: compare the learned controller against the comparison algorithm and, where possible, a simple rule/random baseline.
- Validation: repeat evaluation with fixed unseen seeds/scenarios to check generalization.

8. Expected Interpretation

Expected result: PPO should usually achieve a better energy-comfort trade-off and smoother convergence than REINFORCE. Report actual measured energy savings and comfort violations from the experiment.

9. Conclusion

The experiment demonstrates how PPO vs REINFORCE can be applied to smart building HVAC. The final conclusion should be based on the actual recorded results, not assumed values. A Level 4 submission should clearly connect the observed metrics to algorithm behavior, explain convergence or instability, and justify the selected method.

10. Rubric Coverage

Understanding of Concepts (25%): RL formulation and algorithm explained. Experimental Design (30%): environment, network, training and comparison systematically defined. Use of Tools (20%): Python with TensorFlow/PyTorch and appropriate simulator/data. Analysis of Results (15%): metrics, convergence and validation. Documentation (10%): architecture, implementation, results and conclusion.

5. TRPO for Industrial Robotic Arm

Experiment Objective

Design and implement a policy-based RL solution for automated manufacturing. The experiment uses Trust Region Policy Optimization and evaluates the specified performance measures. The design is structured to satisfy the Level 4 rubric: correct concepts, systematic implementation, appropriate tools, quantitative evaluation and complete documentation.

1. RL Environment Design

- State: joint positions/velocities, end-effector pose, target pose, force/torque and assembly stage.
- Actions: continuous joint or end-effector control commands.
- Reward: positive precision/completion reward and penalties for position error, excessive force, collision risk, cycle time and energy.
- TRPO improves the policy while constraining the change between old and new policies using a KL-divergence trust region.

2. Policy-Based Architecture

Environment State → Policy Network (Actor) → Action → Environment Transition → Reward → Advantage/Return Estimation → Policy Update. Where required, a critic network estimates V(s) and improves the policy-gradient update.

3. Experimental Procedure

- Create or configure the simulation environment and define observation/action spaces.
- Normalize state features and initialize actor/critic networks with reproducible random seeds.
- Train the selected algorithm for a fixed number of episodes/steps and record policy stability, precision, convergence speed, production efficiency.

- Run the same environment settings for the comparison algorithm and use identical evaluation episodes.
- Plot reward and task-specific metrics against training episodes and report mean and standard deviation over multiple seeds where possible.

4. Implementation Skeleton

```
# TRPO core idea (conceptual pseudocode)
old_policy = policy.copy()
collect_trajectories(old_policy)
estimate_advantages()
# maximize surrogate objective
# subject to KL(old_policy || new_policy) <= delta
# use conjugate-gradient search + line search
update_policy_with_trust_region()
```

5. Algorithm Comparison

TRPO is designed to prevent excessively large policy updates. This is valuable for robotic control where unstable updates can produce unsafe actions. Precision should be measured by end-effector error and successful assembly rate.

6. Tools and Experimental Configuration

Use a simulated robot such as a 2D arm, PyBullet or MuJoCo environment. Record end-effector error, collision count, reward, episodes to convergence, cycle time and successful assemblies.

7. Results to Record

- Primary metrics: policy stability, precision, convergence speed, production efficiency.
- Training reward curve: episode/step on the x-axis and mean reward on the y-axis.
- Convergence: identify the point at which the moving-average reward becomes relatively stable.
- Baseline comparison: compare the learned controller against the comparison algorithm and, where possible, a simple rule/random baseline.
- Validation: repeat evaluation with fixed unseen seeds/scenarios to check generalization.

8. Expected Interpretation

Expected result: TRPO should show stable policy improvement and controlled updates. The experiment should demonstrate whether better stability translates into improved precision and production efficiency.

9. Conclusion

The experiment demonstrates how Trust Region Policy Optimization can be applied to automated manufacturing. The final conclusion should be based on the actual recorded results, not assumed values. A Level 4 submission should clearly connect the observed metrics to algorithm behavior, explain convergence or instability, and justify the selected method.

10. Rubric Coverage

Understanding of Concepts (25%): RL formulation and algorithm explained. Experimental Design (30%): environment, network, training and comparison systematically defined. Use of Tools (20%): Python with TensorFlow/PyTorch and appropriate simulator/data. Analysis of Results (15%): metrics, convergence and validation. Documentation (10%): architecture, implementation, results and conclusion.

6. Policy-Based RL for Adaptive Healthcare Treatment

Experiment Objective

Design and implement a policy-based RL solution for simulated patient treatment. The experiment uses A2C vs PPO and evaluates the specified performance measures. The design is structured to satisfy the Level 4 rubric: correct concepts, systematic implementation, appropriate tools, quantitative evaluation and complete documentation.

1. RL Environment Design

- State: simulated patient health indicators, treatment history, current symptom score, adherence and time step.
- Actions: choose among clinically abstract treatment levels or maintain/review treatment. The simulation must enforce safe action limits.
- Reward: improvement toward target health state minus adverse-event, instability and unnecessary-treatment penalties.
- Compare A2C and PPO using the same simulated patient population and episode budget.

2. Policy-Based Architecture

Environment State → Policy Network (Actor) → Action → Environment Transition → Reward → Advantage/Return Estimation → Policy Update. Where required, a critic network estimates V(s) and improves the policy-gradient update.

3. Experimental Procedure

- Create or configure the simulation environment and define observation/action spaces.
- Normalize state features and initialize actor/critic networks with reproducible random seeds.

- Train the selected algorithm for a fixed number of episodes/steps and record treatment effectiveness, cumulative reward, learning stability.
- Run the same environment settings for the comparison algorithm and use identical evaluation episodes.
- Plot reward and task-specific metrics against training episodes and report mean and standard deviation over multiple seeds where possible.

4. Implementation Skeleton

```
import tensorflow as tf
from tensorflow.keras import layers

actor = tf.keras.Sequential([
   layers.Input((6,)), layers.Dense(64, activation="relu"),
   layers.Dense(32, activation="relu"), layers.Dense(3, activation="softmax")
])
critic = tf.keras.Sequential([
   layers.Input((6,)), layers.Dense(64, activation="relu"),
   layers.Dense(1)
])
```

5. Algorithm Comparison

A2C uses an actor and critic with an advantage estimate, while PPO adds a clipped policy update. PPO may be more stable under larger policy changes. Treatment effectiveness must be measured in the simulator using clinically meaningful target achievement rather than reward alone.

6. Tools and Experimental Configuration

Use a synthetic patient simulator; do not use real patient data for an uncontrolled experiment. Compare mean cumulative reward, target-state achievement, adverse-event proxy rate, episode stability and convergence.

7. Results to Record

- Primary metrics: treatment effectiveness, cumulative reward, learning stability.
- Training reward curve: episode/step on the x-axis and mean reward on the y-axis.
- Convergence: identify the point at which the moving-average reward becomes relatively stable.
- Baseline comparison: compare the learned controller against the comparison algorithm and, where possible, a simple rule/random baseline.

- Validation: repeat evaluation with fixed unseen seeds/scenarios to check generalization.

8. Expected Interpretation

Expected result: PPO is expected to provide stable policy updates, while A2C can learn efficiently with lower implementation complexity. The experiment is a simulation study and does not constitute medical treatment advice.

9. Conclusion

The experiment demonstrates how A2C vs PPO can be applied to simulated patient treatment. The final conclusion should be based on the actual recorded results, not assumed values. A Level 4 submission should clearly connect the observed metrics to algorithm behavior, explain convergence or instability, and justify the selected method.

10. Rubric Coverage

Understanding of Concepts (25%): RL formulation and algorithm explained. Experimental Design (30%): environment, network, training and comparison systematically defined. Use of Tools (20%): Python with TensorFlow/PyTorch and appropriate simulator/data. Analysis of Results (15%): metrics, convergence and validation. Documentation (10%): architecture, implementation, results and conclusion.

7. DDPG/PPO for Autonomous Drone Navigation

Experiment Objective

Design and implement a policy-based RL solution for autonomous drone. The experiment uses PPO for dynamic navigation and evaluates the specified performance measures. The design is structured to satisfy the Level 4 rubric: correct concepts, systematic implementation, appropriate tools, quantitative evaluation and complete documentation.

1. RL Environment Design

- State: position, velocity, heading, distance/direction to goal, obstacle distances, wind estimate and battery level.
- Actions: continuous velocity/heading control or discrete safe movement commands.
- Reward: progress toward destination, successful arrival and energy efficiency; penalties for collision, obstacle proximity, excessive control and delay.
- PPO is suitable for stable policy learning; DDPG can be selected when continuous action control is required.

2. Policy-Based Architecture

Environment State → Policy Network (Actor) → Action → Environment Transition → Reward → Advantage/Return Estimation → Policy Update. Where required, a critic network estimates V(s) and improves the policy-gradient update.

3. Experimental Procedure

- Create or configure the simulation environment and define observation/action spaces.

- Normalize state features and initialize actor/critic networks with reproducible random seeds.
- Train the selected algorithm for a fixed number of episodes/steps and record navigation accuracy, obstacle avoidance, energy consumption, policy convergence.
- Run the same environment settings for the comparison algorithm and use identical evaluation episodes.
- Plot reward and task-specific metrics against training episodes and report mean and standard deviation over multiple seeds where possible.

4. Implementation Skeleton

```
import tensorflow as tf
from tensorflow.keras import layers

actor = tf.keras.Sequential([
   layers.Input((10,)), layers.Dense(128, activation="tanh"),
   layers.Dense(64, activation="tanh"), layers.Dense(4, activation="softmax")
])
critic = tf.keras.Sequential([
   layers.Input((10,)), layers.Dense(128, activation="tanh"),
   layers.Dense(1)
])
```

5. Algorithm Comparison

Dynamic environments require the policy to react to changing obstacles. Training should use randomized obstacle layouts and sensor noise. Exploration must occur primarily in simulation; real drones need safety constraints and emergency return behavior.

6. Tools and Experimental Configuration

Use PyBullet, AirSim or another simulator. Evaluate final-position error, collision rate, route length, energy use, successful navigation and reward convergence.

7. Results to Record

- Primary metrics: navigation accuracy, obstacle avoidance, energy consumption, policy convergence.
- Training reward curve: episode/step on the x-axis and mean reward on the y-axis.
- Convergence: identify the point at which the moving-average reward becomes relatively stable.
- Baseline comparison: compare the learned controller against the comparison algorithm and, where possible, a simple rule/random baseline.

- Validation: repeat evaluation with fixed unseen seeds/scenarios to check generalization.

8. Expected Interpretation

Expected result: the policy should learn shorter and safer paths than random/control baselines. Exact navigation accuracy and energy values must come from the executed experiment.

9. Conclusion

The experiment demonstrates how PPO for dynamic navigation can be applied to autonomous drone. The final conclusion should be based on the actual recorded results, not assumed values. A Level 4 submission should clearly connect the observed metrics to algorithm behavior, explain convergence or instability, and justify the selected method.

10. Rubric Coverage

Understanding of Concepts (25%): RL formulation and algorithm explained. Experimental Design (30%): environment, network, training and comparison systematically defined. Use of Tools (20%): Python with TensorFlow/PyTorch and appropriate simulator/data. Analysis of Results (15%): metrics, convergence and validation. Documentation (10%): architecture, implementation, results and conclusion.

8. A3C for Dynamic Cloud Resource Allocation

Experiment Objective

Design and implement a policy-based RL solution for cloud computing. The experiment uses A3C vs Vanilla Policy Gradient and evaluates the specified performance measures. The design is structured to satisfy the Level 4 rubric: correct concepts, systematic implementation, appropriate tools, quantitative evaluation and complete documentation.

1. RL Environment Design

- State: CPU utilization, memory utilization, queue length, request rate, active instances and response time.
- Actions: allocate/release virtual machines or change resource allocation level.
- Reward: positive resource efficiency and completed requests; penalties for response time, overload, underutilization and unnecessary instances.
- A3C uses multiple asynchronous workers that explore different environment trajectories and update shared policy/value parameters.

2. Policy-Based Architecture

Environment State → Policy Network (Actor) → Action → Environment Transition → Reward → Advantage/Return Estimation → Policy Update. Where required, a critic network estimates V(s) and improves the policy-gradient update.

3. Experimental Procedure

- Create or configure the simulation environment and define observation/action spaces.
- Normalize state features and initialize actor/critic networks with reproducible random seeds.

- Train the selected algorithm for a fixed number of episodes/steps and record response time, CPU utilization, resource efficiency.
- Run the same environment settings for the comparison algorithm and use identical evaluation episodes.
- Plot reward and task-specific metrics against training episodes and report mean and standard deviation over multiple seeds where possible.

4. Implementation Skeleton

```
import tensorflow as tf
from tensorflow.keras import layers

actor = tf.keras.Sequential([
   layers.Input((6,)), layers.Dense(64, activation="relu"),
   layers.Dense(32, activation="relu"), layers.Dense(4, activation="softmax")
])
critic = tf.keras.Sequential([
   layers.Input((6,)), layers.Dense(64, activation="relu"),
   layers.Dense(1)
])

# A3C idea:
# Worker -> collect trajectory -> calculate gradients
#       -> asynchronously update shared actor/critic
```

5. Algorithm Comparison

A3C can improve exploration by running independent workers with different experiences. Vanilla Policy Gradient uses complete trajectory returns and can show high variance. Compare response time, CPU utilization, resource efficiency and reward stability.

6. Tools and Experimental Configuration

Implement a cloud simulator with variable request loads. Workers can run separate environment copies. Record average response time, utilization, rejected requests, allocated resources and reward.

7. Results to Record

- Primary metrics: response time, CPU utilization, resource efficiency.
- Training reward curve: episode/step on the x-axis and mean reward on the y-axis.

- Convergence: identify the point at which the moving-average reward becomes relatively stable.
- Baseline comparison: compare the learned controller against the comparison algorithm and, where possible, a simple rule/random baseline.
- Validation: repeat evaluation with fixed unseen seeds/scenarios to check generalization.

8. Expected Interpretation

Expected result: A3C should generally learn a more stable allocation policy and adapt to workload changes, while avoiding excessive resource allocation.

9. Conclusion

The experiment demonstrates how A3C vs Vanilla Policy Gradient can be applied to cloud computing. The final conclusion should be based on the actual recorded results, not assumed values. A Level 4 submission should clearly connect the observed metrics to algorithm behavior, explain convergence or instability, and justify the selected method.

10. Rubric Coverage

Understanding of Concepts (25%): RL formulation and algorithm explained. Experimental Design (30%): environment, network, training and comparison systematically defined. Use of Tools (20%): Python with TensorFlow/PyTorch and appropriate simulator/data. Analysis of Results (15%): metrics, convergence and validation. Documentation (10%): architecture, implementation, results and conclusion.

9. Policy Gradient Predictive Maintenance

Experiment Objective

Design and implement a policy-based RL solution for industrial machine maintenance. The experiment uses REINFORCE vs PPO and evaluates the specified performance measures. The design is structured to satisfy the Level 4 rubric: correct concepts, systematic implementation, appropriate tools, quantitative evaluation and complete documentation.

1. RL Environment Design

- State: vibration, temperature, pressure, operating hours, health score, recent maintenance and failure indicators.
- Actions: continue operation, inspect, schedule maintenance, reduce load or replace an approved component.
- Reward: minimize maintenance cost and downtime while strongly penalizing unexpected failure. A large failure penalty prevents the policy from choosing risky operation for small short-term savings.
- Policy network outputs probabilities over maintenance decisions.

2. Policy-Based Architecture

Environment State → Policy Network (Actor) → Action → Environment Transition → Reward → Advantage/Return Estimation → Policy Update. Where required, a critic network estimates V(s) and improves the policy-gradient update.

3. Experimental Procedure

- Create or configure the simulation environment and define observation/action spaces.
- Normalize state features and initialize actor/critic networks with reproducible random seeds.

- Train the selected algorithm for a fixed number of episodes/steps and record maintenance cost, machine downtime, reward and REINFORCE-vs-PPO performance.
- Run the same environment settings for the comparison algorithm and use identical evaluation episodes.
- Plot reward and task-specific metrics against training episodes and report mean and standard deviation over multiple seeds where possible.

4. Implementation Skeleton

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense

policy = Sequential([
    Dense(64, activation="relu", input_shape=(7,)),
    Dense(32, activation="relu"),
    Dense(5, activation="softmax")
])

# Train with REINFORCE:
# loss = -sum(log(policy_action) * discounted_return)

# PPO uses the same policy representation but clips
# the probability ratio during policy updates.
```

5. Algorithm Comparison

REINFORCE directly estimates policy gradients but may have high variance. PPO controls update size and commonly provides more stable optimization. Maintenance rewards must represent the true business objective, including downtime and failure risk.

6. Tools and Experimental Configuration

Use simulated sensor trajectories or a public predictive-maintenance dataset converted into an MDP. Compare total maintenance cost, downtime, unexpected failures, episode reward and convergence.

7. Results to Record

- Primary metrics: maintenance cost, machine downtime, reward and REINFORCE-vs-PPO performance.

- Training reward curve: episode/step on the x-axis and mean reward on the y-axis.
- Convergence: identify the point at which the moving-average reward becomes relatively stable.
- Baseline comparison: compare the learned controller against the comparison algorithm and, where possible, a simple rule/random baseline.
- Validation: repeat evaluation with fixed unseen seeds/scenarios to check generalization.

8. Expected Interpretation

Expected result: a good policy should schedule maintenance before costly failures while avoiding unnecessary maintenance. PPO is expected to converge more smoothly than REINFORCE in many settings.

9. Conclusion

The experiment demonstrates how REINFORCE vs PPO can be applied to industrial machine maintenance. The final conclusion should be based on the actual recorded results, not assumed values. A Level 4 submission should clearly connect the observed metrics to algorithm behavior, explain convergence or instability, and justify the selected method.

10. Rubric Coverage

Understanding of Concepts (25%): RL formulation and algorithm explained. Experimental Design (30%): environment, network, training and comparison systematically defined. Use of Tools (20%): Python with TensorFlow/PyTorch and appropriate simulator/data. Analysis of Results (15%): metrics, convergence and validation. Documentation (10%): architecture, implementation, results and conclusion.

10. PPO/TRPO for Autonomous Lane-Keeping

Experiment Objective

Design and implement a policy-based RL solution for autonomous driving. The experiment uses PPO vs A2C and evaluates the specified performance measures. The design is structured to satisfy the Level 4 rubric: correct concepts, systematic implementation, appropriate tools, quantitative evaluation and complete documentation.

1. RL Environment Design

- State: lateral lane deviation, heading error, vehicle speed, road curvature, steering history and lane-boundary distances.
- Actions: steering control or bounded steering commands.
- Reward: high reward for staying near lane center and following heading; penalties for lane deviation, unsafe steering, oscillation and lane departure.
- Use PPO or TRPO as the primary controller and A2C as the comparison baseline.

2. Policy-Based Architecture

Environment State → Policy Network (Actor) → Action → Environment Transition → Reward → Advantage/Return Estimation → Policy Update. Where required, a critic network estimates V(s) and improves the policy-gradient update.

3. Experimental Procedure

- Create or configure the simulation environment and define observation/action spaces.
- Normalize state features and initialize actor/critic networks with reproducible random seeds.

- Train the selected algorithm for a fixed number of episodes/steps and record lane deviation, safety, reward convergence, training efficiency.
- Run the same environment settings for the comparison algorithm and use identical evaluation episodes.
- Plot reward and task-specific metrics against training episodes and report mean and standard deviation over multiple seeds where possible.

4. Implementation Skeleton

```
import tensorflow as tf
from tensorflow.keras import layers

actor = tf.keras.Sequential([
   layers.Input((6,)), layers.Dense(128, activation="tanh"),
   layers.Dense(64, activation="tanh"), layers.Dense(3, activation="softmax")
])
critic = tf.keras.Sequential([
   layers.Input((6,)), layers.Dense(128, activation="tanh"),
   layers.Dense(1)
])

# PPO objective uses clipped probability ratios
# to keep each update within a controlled range.
```

5. Algorithm Comparison

PPO is selected because stable policy updates are important in continuous control. A2C is a useful baseline because it also uses actor and critic networks but does not use PPO's clipped objective.

6. Tools and Experimental Configuration

Use a driving simulator such as CARLA or a simplified lane-keeping environment. Measure mean lateral deviation, lane departure rate, reward, convergence episodes and training time.

7. Results to Record

- Primary metrics: lane deviation, safety, reward convergence, training efficiency.
- Training reward curve: episode/step on the x-axis and mean reward on the y-axis.
- Convergence: identify the point at which the moving-average reward becomes relatively stable.

- Baseline comparison: compare the learned controller against the comparison algorithm and, where possible, a simple rule/random baseline.
- Validation: repeat evaluation with fixed unseen seeds/scenarios to check generalization.

8. Expected Interpretation

Expected result: PPO should provide stable convergence and low lane deviation, while A2C can provide a useful baseline. Safety evaluation must include lane departures and unstable steering, not only reward.

9. Conclusion

The experiment demonstrates how PPO vs A2C can be applied to autonomous driving. The final conclusion should be based on the actual recorded results, not assumed values. A Level 4 submission should clearly connect the observed metrics to algorithm behavior, explain convergence or instability, and justify the selected method.

10. Rubric Coverage

Understanding of Concepts (25%): RL formulation and algorithm explained. Experimental Design (30%): environment, network, training and comparison systematically defined. Use of Tools (20%): Python with TensorFlow/PyTorch and appropriate simulator/data. Analysis of Results (15%): metrics, convergence and validation. Documentation (10%): architecture, implementation, results and conclusion.